

Paging

1 Core Idea

Definition

Paging is a virtual-memory technique that divides a process's virtual address space into fixed-size **pages** and physical memory into same-size **page frames**.

Programs generate **virtual addresses**. With paging, these virtual addresses are not sent directly to memory. They first go through the **Memory Management Unit (MMU)**, which translates them into physical addresses.

Basic Mapping

$$\text{virtual address} \xrightarrow{\text{MMU}} \text{physical address}$$

2 MMU

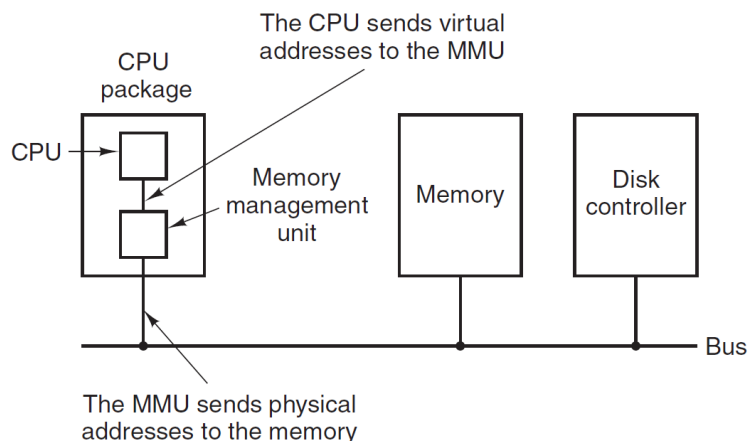


Figure 3-8. The position and function of the MMU. Here the MMU is shown as being a part of the CPU chip because it commonly is nowadays. However, logically it could be a separate chip and was years ago.

What the MMU Does

The CPU sends virtual addresses to the MMU. The MMU translates them and sends physical addresses to memory.

Component	Role
CPU	Generates virtual addresses while executing instructions.
MMU	Maps virtual addresses to physical addresses.
Memory	Sees only physical addresses. It does not know about virtual addresses.
Disk	Stores pages that are not currently in RAM.

3 Virtual Pages and Page Frames

Key Terms

Virtual page: a fixed-size block of virtual addresses.

Page frame: a fixed-size block of physical memory that can hold one page.

Pages and page frames normally have the same size. In the example below:

- virtual address space = 64 KB;
- physical memory = 32 KB;
- page size = 4 KB;
- number of virtual pages = $64/4 = 16$;
- number of physical frames = $32/4 = 8$.

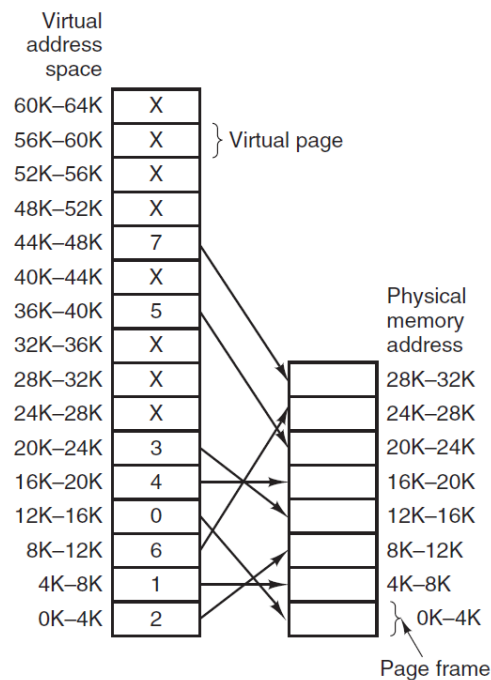


Figure 3-9. The relation between virtual addresses and physical memory addresses is given by the **page table**. Every page begins on a multiple of 4096 and ends 4095 addresses higher, so 4K-8K really means 4096-8191 and 8K to 12K means 8192-12287.

4 Reading Page Ranges

The notation $0K - 4K$ means virtual or physical addresses 0 through 4095. It does not include address 4096.

Range	Actual addresses
0K – 4K	0 to 4095
4K – 8K	4096 to 8191
8K – 12K	8192 to 12287

Why Powers of Two Help

With 4-KB pages, each page has $4096 = 2^{12}$ addresses. This makes the lower 12 bits of an address the offset inside the page.

5 Page Table

Page Table

A **page table** records where each virtual page is located in physical memory, or whether it is absent.

In Fig. 3-9, each virtual page entry contains either:

- a page-frame number, if the page is present in RAM;
- X , if the page is not currently mapped to RAM.

Present/Absent Bit

Real hardware uses a present/absent bit. If the bit says “absent,” a reference to that page causes a page fault.

6 Address Translation Formula

Let:

- V = virtual address;
- P = virtual page number;
- O = offset inside the page;
- F = physical page-frame number;
- page size = 4096 bytes.

Paging Translation

$$\begin{aligned}
 P &= \lfloor V/4096 \rfloor \\
 O &= V \bmod 4096 \\
 \text{physical address} &= F \times 4096 + O
 \end{aligned}$$

7 Example 1: Virtual Address 0

Problem

Translate virtual address 0. From Fig. 3-9, virtual page 0 maps to page frame 2.

Solution

$$P = \lfloor 0/4096 \rfloor = 0$$

$$O = 0 \bmod 4096 = 0$$

Virtual page 0 maps to frame 2, so $F = 2$.

$$\text{physical address} = 2 \times 4096 + 0 = 8192$$

Answer

MOV REG,0 uses physical address 8192.

8 Example 2: Virtual Address 8192

Problem

Translate virtual address 8192. From Fig. 3-9, virtual page 2 maps to page frame 6.

Solution

$$P = \lfloor 8192/4096 \rfloor = 2$$

$$O = 8192 \bmod 4096 = 0$$

Virtual page 2 maps to frame 6, so $F = 6$.

$$\text{physical address} = 6 \times 4096 + 0 = 24576$$

Answer

MOV REG,8192 is effectively transformed into MOV REG,24576.

9 Example 3: Virtual Address 20500

Problem

Translate virtual address 20500. This address is in virtual page 5, and virtual page 5 maps to page frame 3.

Solution

$$P = \lfloor 20500/4096 \rfloor = 5$$

$$O = 20500 - (5 \times 4096)$$

$$O = 20500 - 20480 = 20$$

Virtual page 5 maps to frame 3, so $F = 3$.

$$\text{physical address} = 3 \times 4096 + 20 = 12288 + 20 = 12308$$

Answer

Virtual address 20500 maps to physical address 12308.

10 Unmapped Pages and Page Faults

Only 8 physical frames exist in this example, so only 8 of the 16 virtual pages can be in RAM at once. Entries marked X are absent.

Page Fault

A **page fault** occurs when the MMU sees that the referenced virtual page is not currently mapped to a physical frame.

Example:

```
MOV REG, 32780
```

Virtual address 32780 is 12 bytes into virtual page 8:

$$32780 = 32768 + 12$$

But virtual page 8 is marked X , so the MMU traps to the operating system.

11 Handling a Page Fault

When a valid page is absent, the operating system handles the fault.

Step	Action
1	Choose a page frame to reuse, often one that has not been used recently.
2	If the old page in that frame was modified, write it back to disk.
3	Read the needed page from disk into the freed frame.
4	Update the page table entry.
5	Restart the instruction that caused the page fault.

Example Result

If virtual page 8 is loaded into physical frame 1, then virtual address 32780 maps to $4096 + 12 = 4108$.

12 Inside the MMU

For a 16-bit virtual address with 4-KB pages:

- page size = $4096 = 2^{12}$, so the offset uses 12 bits;
- there are 16 virtual pages, so the page number uses 4 bits;
- physical memory is 32 KB = 8 frames, so the frame number uses 3 bits.

Address part	Meaning
High 4 bits	Virtual page number.
Low 12 bits	Offset inside the page.
Page table result	Physical frame number.
Physical address	Frame number plus unchanged offset.

Key Detail

The offset is copied unchanged. Only the page number is translated into a frame number.

13 Summary

Concept	Meaning
Paging	Divide virtual memory into pages and physical memory into frames.
MMU	Hardware that translates virtual addresses to physical addresses.
Page table	Data structure that maps virtual pages to frames.
Offset	Position inside a page; copied unchanged during translation.
Present/absent bit	Says whether the page is currently in RAM.
Page fault	Trap caused by referencing a valid page that is absent from RAM.